School of Future Tech

Case Study Report

Library Book Issue System

Your Case Study Title

By

SIMRAN SUTAR

ROLL NO:

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Problem Statement / Case Study Design
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study
5. Problem Statement / Case Study Implementation Details and Snapshots
6. Problem Statement / Case Study Results and Conclusion
7. References

1. Introduction to the Case Study

Libraries are important resources for educational institutions, providing students and faculty with access to books and learning materials. Managing library records manually can be time-consuming and prone to errors, especially when tracking issued and returned books.

This case study focuses on developing a **Library Book Issue System using C++** that helps manage books in a simple and efficient way. The system allows users to add new books, issue books to readers, return issued books, and display the list of available books.

The program demonstrates how programming concepts such as **structures, arrays, file handling, and menu-driven programming** can be applied to build a real-world record management system. By storing book data in a file, the system ensures that records are saved and can be accessed whenever the program runs again.

2. Problem Statement / Case Background (Abstract)

Background

Libraries often maintain records of books and their availability. In small libraries or learning environments, this process may be handled manually, which can lead to mistakes such as losing track of issued books or incorrect availability status.

A digital system can simplify these tasks by automatically updating records whenever a book is added, issued, or returned.

Abstract

This case study presents the design and implementation of a **Library Book Issue System using C++**. The system stores book information such as **book ID, title, and issue status** in a file. Users interact with the system through a **menu-driven interface** that provides options to add books, issue books, return books, and display available books.

The system uses **structures to represent book records**, **vectors or arrays to store multiple records**, and **file handling to maintain data persistence**. The implementation demonstrates

how basic programming techniques can be used to develop a simple yet effective library management application.

3. Case Study Design

The Library Book Issue System is designed as a **menu-driven application** consisting of several functional modules.

Add Book

This module allows the user to add a new book to the library by entering the **book ID and book title**. The book is then stored in the system with an availability status.

Issue Book

In this module, the user enters the **book ID** of the book to issue. If the book is available, the system updates its status.

Return Book

This module allows users to return a previously issued book. The system updates the book status back to **available**.

Show Available Books

This module displays the list of books that are currently available in the library.

Data Storage

All book records are stored in a **text file (books.txt)** so that the data remains saved even after the program is closed.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

The implementation uses several programming concepts and methods.

Data Structure

A **structure (struct)** is used to represent each book record containing:

- Book ID
- Book Title
- Issue Status (issued or available)

Example structure:

```
struct Book
{
  int id;
  string title;
  bool issued;
};
```

File Handling

The system uses **file input/output operations** to store and retrieve book data from the file **books.txt**.

Functions used include:

- `ifstream` for reading data
- `ofstream` for writing data

Menu Driven Programming

The program uses a **loop with a switch statement** to display options repeatedly until the user chooses to exit.

Example algorithm for the menu:

1. Display menu options
2. Take user input
3. Perform the selected operation
4. Update the file if necessary
5. Repeat until user selects exit

Technologies Used

Programming Language:

- C++

Libraries Used:

- `iostream` for input and output
- `fstream` for file handling
- `vector` for storing multiple book records

Development Environment:

- Visual Studio Code / CodeBlocks / any C++ compiler

Data Storage:

- Text file (`books.txt`)

5. Problem Statement / Case Study Implementation Details and Snapshots

The implementation is written in a single C++ file named:

library.cpp

The program performs the following steps:

Loading Data

When the program starts, it reads the book records stored in **books.txt** and loads them into memory.

User Menu

The program displays a menu like this:

```
===== Library Menu =====
1. Add Book
2. Issue Book
3. Return Book
4. Show Available Books
5. Exit
```

Adding a Book

The user enters:

- Book ID
- Book Title

The system saves this information to the file.

Issuing a Book

The user enters the **book ID**, and if the book exists and is available, it is marked as issued.

Returning a Book

The system changes the book status from **issued** to **available**.

Displaying Available Books

The program prints all books that are not currently issued.

Snapshots to Include

You can add screenshots of:

- Running program in terminal
- Adding a new book
- Issuing a book
- Returning a book
- Displaying available books

6. Problem Statement / Case Study Results and Conclusion

Results

The developed Library Book Issue System successfully performs the following operations:

- Adding new books to the library database
- Issuing books to users
- Returning books to the library
- Displaying available books
- Saving records in a file for future use

The system is simple, efficient, and demonstrates how programming concepts can be applied to manage real-world data.

Conclusion

This case study demonstrates the development of a **basic Library Book Issue System using C++**. The project applies important programming concepts such as **structures, file handling, vectors, and menu-driven programming**.

Although the system is simple, it provides a strong foundation for building more advanced systems such as full **Library Management Systems with user accounts, due dates, and graphical interfaces**.

The case study highlights how software solutions can improve record management and reduce manual effort in everyday tasks.

7. References

1. C++ Programming Language documentation
2. Tutorials on C++ file handling and structures
3. Standard C++ libraries documentation (`iostream`, `fstream`, `vector`)
4. Programming learning resources such as GeeksforGeeks and W3Schools
5. Classroom materials and programming lectures on C++